

Digital Modeling of a Guitar Amplifier Tone Stack



Digital Modeling of a Guitar Amplifier Tone Stack

Site: **Ampbooks**

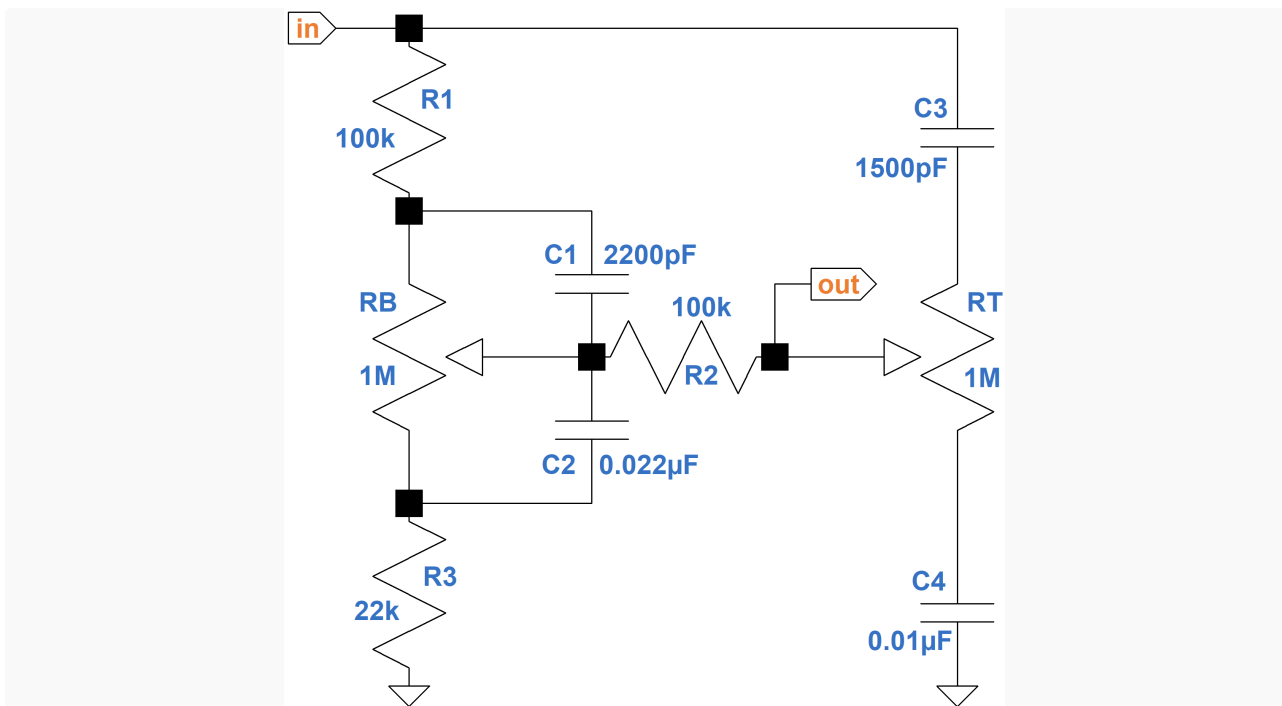
Saved Today at 21:07

...

"We exhaustively worked out the equations for all the tone stacks by evaluating the mesh and node equations. Fortunately we have computers that handled the polynomial reduction but it was still a Herculean effort." ¹ -Fractal Audio Systems

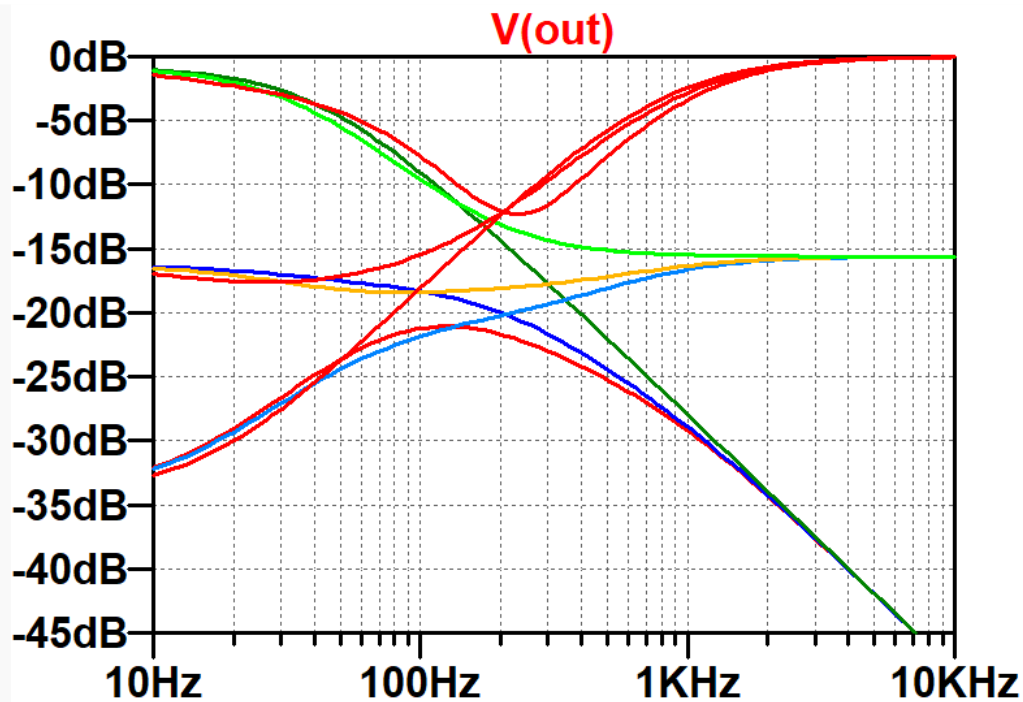
Here we describe the procedure most commonly used to create a digital model of a guitar amplifier tone stack. The formulas are quite lengthy - if you are reading this page with a smartphone, please rotate your display for landscape (horizontal) mode so that they fit on the screen.

David Yeh and Julius Smith have already published a DSP model of a classic Fender Bassman stack.² Rather than embellish upon their work here, let's model another popular circuit, the James tone stack as found in the Orange Graphic Mk II (1972 version), where it is sandwiched between the first preamp stage and a gain control.



The bass control is on the left. Treble is on the right. The Mk II driving circuit has an output impedance of 50kΩ and the driven circuit has a 1MΩ input impedance, which have a slight impact on performance. We will ignore these details to shorten the equations, which for tone stacks are very large.

Here are the results of a SPICE simulation³ of the analog circuit.



The controls are set to minimum, 15-percent resistance, and maximum. This plot represents a benchmark - the digital model should match these results.

The Analog Model

Let Z_1 represent the impedance formed by the resistance of the bass potentiometer R_B above the wiper in parallel with the reactance X_{C1} of the capacitor C_1 . Similarly, let Z_2 be the resistance below the wiper in parallel with the reactance of C_2 . We will denote the reactances of C_3 and C_4 as X_{C3} and X_{C4} , respectively.

Let's use [Kirchhoff's Voltage Law](#) around three loops.

1. from ground, through the guitar signal voltage source driving the input, through C_3 , then the treble control R_T , through C_4 , and back to ground,
2. R_1 , C_3 , $(1-PT)R_T$, R_2 , Z_1 , where PT , which ranges from 0.0 to 1.0, is the position of the treble control and $(1-PT)R_T$ is the resistance above the wiper, and
3. R_3 , Z_2 , R_2 , $PT R_T$, and C_4 , where $PT R_T$ is the resistance below the treble control wiper.

According to Kirchhoff, here are the three loop equations:

$$X_{C3}(i_1 + i_2) + (1 - P_T)R_T(i_1 + i_2) + P_TR_T(i_1 + i_3) + X_{C4}(i_1 + i_3) = V_{in}$$

$$R_1i_2 + X_{C3}(i_1 + i_2) + (1 - P_T)R_T(i_1 + i_2) + R_2(i_2 - i_3) + Z_1i_2 = 0$$

$$R_3i_3 + Z_2i_3 + R_2(i_3 - i_2) + P_TR_T(i_1 + i_3) + X_{C4}(i_1 + i_3) = 0$$

According to Ohm's Law, the output voltage is $V_{out} = (P_TR_T + X_{C4})(i_1 + i_3)$.

Gathering the currents, the four equations become

$$(X_{C3} + R_T + X_{C4})i_1 + [X_{C3} + (1 - P_T)R_T]i_2 + (P_TR_T + X_{C4})i_3 = V_{in}$$

$$[X_{C3} + (1 - P_T)R_T]i_1 + [R_1 + X_{C3} + (1 - P_T)R_T + R_2 + Z_1]i_2 - R_2i_3 = 0$$

$$(P_TR_T + X_{C4})i_1 - R_2i_2 + (R_3 + Z_2 + R_2 + P_TR_T + X_{C4})i_3 = 0$$

$$V_{out} = (P_TR_T + X_{C4})i_1 + (P_TR_T + X_{C4})i_3$$

If we use matrices and vectors, these separate equations can be combined into a single formula that can be solved using software, which because of the complexity of the process is the only practical approach. Here is a matrix of impedances, a vector of currents, and a vector of voltages, i.e. Ohm's Law in matrix form:

$$ZI = V$$

where $Z =$

$$\begin{bmatrix} X_{C3} + R_T + X_{C4} & X_{C3} + (1 - P_T)R_T & P_T R_T + X_{C4} \\ X_{C3} + (1 - P_T)R_T & R_1 + X_{C3} + (1 - P_T)R_T + R_2 + Z_1 & -R_2 \\ P_T R_T + X_{C4} & -R_2 & R_3 + Z_2 + R_2 + P_T R_T + X_{C4} \end{bmatrix}$$

$$I = \begin{bmatrix} i_1 \\ i_2 \\ i_3 \end{bmatrix}$$

$$V = \begin{bmatrix} V_{in} \\ 0 \\ 0 \end{bmatrix}$$

If we invert the 3 by 3 matrix, which is easily performed in software, then $I = Z^{-1}V$. The output voltage is

$$\begin{aligned} V_{out} &= [(P_T R_T + X_{C4} \quad 0 \quad P_T R_T + X_{C4})] \begin{bmatrix} i_1 \\ i_2 \\ i_3 \end{bmatrix} \\ &= [(P_T R_T + X_{C4} \quad 0 \quad P_T R_T + X_{C4})] I \\ &= [(P_T R_T + X_{C4} \quad 0 \quad P_T R_T + X_{C4})] Z^{-1} V \\ &= [(P_T R_T + X_{C4} \quad 0 \quad P_T R_T + X_{C4})] Z^{-1} \begin{bmatrix} V_{in} \\ 0 \\ 0 \end{bmatrix} \end{aligned}$$

Finally, the voltage gain is the output voltage divided by the input voltage:

$$H(s) = \frac{V_{out}}{V_{in}} = [(P_T R_T + X_{C4} \quad 0 \quad P_T R_T + X_{C4}) Z^{-1}] \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

The actual formula for the inverted matrix Z-1 is enormous, particularly when these substitutions are made:

$$X_{C3} = \frac{1}{C_3 s}$$

$$X_{C4} = \frac{1}{C_4 s}$$

$$Z_1 = (1 - P_B) R_B \| X_{C1} = (1 - P_B) R_B \| \frac{1}{C_1 s} = \frac{1}{\frac{1}{(1 - P_B) R_B} + C_1 s}$$

$$Z_2 = P_B R_B \| X_{C2} = P_B R_B \| \frac{1}{C_2 s} = \frac{1}{\frac{1}{P_B R_B} + C_2 s}$$

The formula for H(s) can be entered into software that can manipulate the vectors and invert the matrix symbolically to solve for the gain. Maple, Mathematica, MathCad and some open source packages easily perform this feat. Here we use MathCad. David Yeh and Julius Smith use Mathematica for the Bassman stack.

There are two ways to solve for the gain symbolically. One is to retain the variable names. If the results include the product of, say, R1 and R2, then the final formula for gain contains, literally, "R1R2". This is the approach used by Yeh and Smith, which is handy if part values are subject to change. The other approach, which will be used here, is to substitute the actual part values. In this case the product of the two 100kΩ resistances is a single number: 1010. This approach is usually less verbose and is handy for modeling a specific tone stack in which only the control positions are subject to change.

Here is the resulting analog model of the James stack using parts values from the Orange Graphic Mk II (1972 version).

$$H(s) = \frac{b_0 + b_1s + b_2s^2 + b_3s^3 + b_4s^4}{a_0 + a_1s + a_2s^2 + a_3s^3 + a_4s^4}$$

where the coefficients depend only on knob positions:

$$\begin{aligned} a_0 &= 9.34 \times 10^{10} \\ a_1 &= (-2.975 \times 10^9)P_B^2 + (3.251 \times 10^9)P_B + (7.948 \times 10^8)P_T + 2.934 \times 10^8 \\ a_2 &= 2.344 \times 10^5 - (7.761 \times 10^6)P_B^2 + (1.885 \times 10^7)P_BP_T + (8.434 \times 10^6)P_B \\ &\quad + (1.593 \times 10^6)P_T - (1.403 \times 10^6)P_T^2 - (1.714 \times 10^7)P_B^2P_T \\ a_3 &= (-33269)P_BP_T^2 + (5667)P_B + (37452)P_BP_T - (5311)P_B^2 + (335.3)(P_T - P_T^2) \\ &\quad - (34433)P_B^2P_T + (30250)P_B^2P_T^2 + 39.6 \\ a_4 &= (7.381)(P_BP_T + P_B^2P_T^2 - P_BP_T^2 - P_B^2P_T) + (0.8712)(P_B - P_B^2) \\ b_0 &= (8.333 \times 10^{10})P_B + 1.833 \times 10^9 \\ b_1 &= (7.083 \times 10^8)P_BP_T - (3.083 \times 10^8)P_B^2 + (4.794 \times 10^8)P_B + (1.558 \times 10^7)P_T \\ &\quad + 2.383 \times 10^7 \\ b_2 &= (844320)P_B - (2.808 \times 10^6)P_B^2P_T + (232280)P_T + (4.464 \times 10^6)P_BP_T \\ &\quad - (754230)P_B^2 - (1.25 \times 10^6)P_BP_T^2 - (27500)P_T^2 + 10010 \\ b_3 &= (220.2)(P_B - P_B^2) + (8310)P_BP_T - (7409)P_B^2P_T + (100.1)P_T + (2750)P_B^2P_T^2 \\ &\quad - (60.5)P_T^2 - (3294.5)P_BP_T^2 \\ b_4 &= (2.202)(P_BP_T - P_B^2P_T) + (1.331)(P_B^2P_T^2 - P_BP_T^2) \end{aligned}$$

Transforming the Analog Model into a Digital Model

"Simulating drive and tone controls is very difficult and laborious. The equations are onerous and complicated, especially for tone stacks." 4 - Fractal Audio Systems

The relationship between the coefficients of the digital filter and the coefficients for the analog filter depends on the sample rate f_s . Let's assume a 48kHz sample rate. Using a bilinear transform we substitute this expression for the analog variable s :

$$s = (2)f_s \frac{1 - z^{-1}}{1 + z^{-1}} = (2)(48000) \frac{1 - z^{-1}}{1 + z^{-1}}$$

Using math software again to simplify the results, the continuous-time system representation is transformed to a discrete-time representation:

$$H(z) = \frac{B_0 + B_1z^{-1} + B_2z^{-2} + B_3z^{-3} + B_4z^{-4}}{A_0 + A_1z^{-1} + A_2z^{-2} + A_3z^{-3} + A_4z^{-4}}$$

where

$$A_0 = a_0 + (96000)a_1 + (9.216 \times 10^9)a_2 + (8.84736 \times 10^{14})a_3 + (8.4934656 \times 10^{19})a_4$$

$$A_1 = 4a_0 + (192000)a_1 - (1.769472 \times 10^{15})a_3 - (3.39738624 \times 10^{20})a_4$$

$$A_2 = 6a_0 - (1.8432 \times 10^{10})a_2 + (5.09607936 \times 10^{20})a_4$$

$$A_3 = 4a_0 - (192000)a_1 + (1.769472 \times 10^{15})a_3 - (3.39738624 \times 10^{20})a_4$$

$$A_4 = a_0 - (96000)a_1 + (9.216 \times 10^9)a_2 - (8.84736 \times 10^{14})a_3 + (8.4934656 \times 10^{19})a_4$$

$$B_0 = b_0 + (96000)b_1 + (9.216 \times 10^9)b_2 + (8.84736 \times 10^{14})b_3 + (8.4934656 \times 10^{19})b_4$$

$$B_1 = 4b_0 + (192000)b_1 - (1.769472 \times 10^{15})b_3 - (3.39738624 \times 10^{20})b_4$$

$$B_2 = 6b_0 - (1.8432 \times 10^{10})b_2 + (5.09607936 \times 10^{20})b_4$$

$$B_3 = 4b_0 - (192000)b_1 + (1.769472 \times 10^{15})b_3 - (3.39738624 \times 10^{20})b_4$$

$$B_4 = b_0 - (96000)b_1 + (9.216 \times 10^9)b_2 - (8.84736 \times 10^{14})b_3 + (8.4934656 \times 10^{19})b_4$$

The frequency response for the digital filter, as a function of z , is the output divided by the input:

$$\frac{Y(z)}{X(z)} = H(z) = \frac{B_0 + B_1z^{-1} + B_2z^{-2} + B_3z^{-3} + B_4z^{-4}}{A_0 + A_1z^{-1} + A_2z^{-2} + A_3z^{-3} + A_4z^{-4}}$$

The relationship between the input and output is therefore

$$\begin{aligned} Y(z)[A_0 + A_1z^{-1} + A_2z^{-2} + A_3z^{-3} + A_4z^{-4}] \\ = X(z)[B_0 + B_1z^{-1} + B_2z^{-2} + B_3z^{-3} + B_4z^{-4}] \end{aligned}$$

Since each z^{-1} represents a delay of 1 sample (z^{-2} is a delay of 2 samples, z^{-3} is a delay of 3 samples, ...), in the time domain we get

$$A_0y[n] + A_1y[n - 1] + A_2y[n - 2] + A_3y[n - 3] + A_4y[n - 4]$$

$$= B_0x[n] + B_1x[n - 1] + B_2x[n - 2] + B_3x[n - 3] + B_4x[n - 4]$$

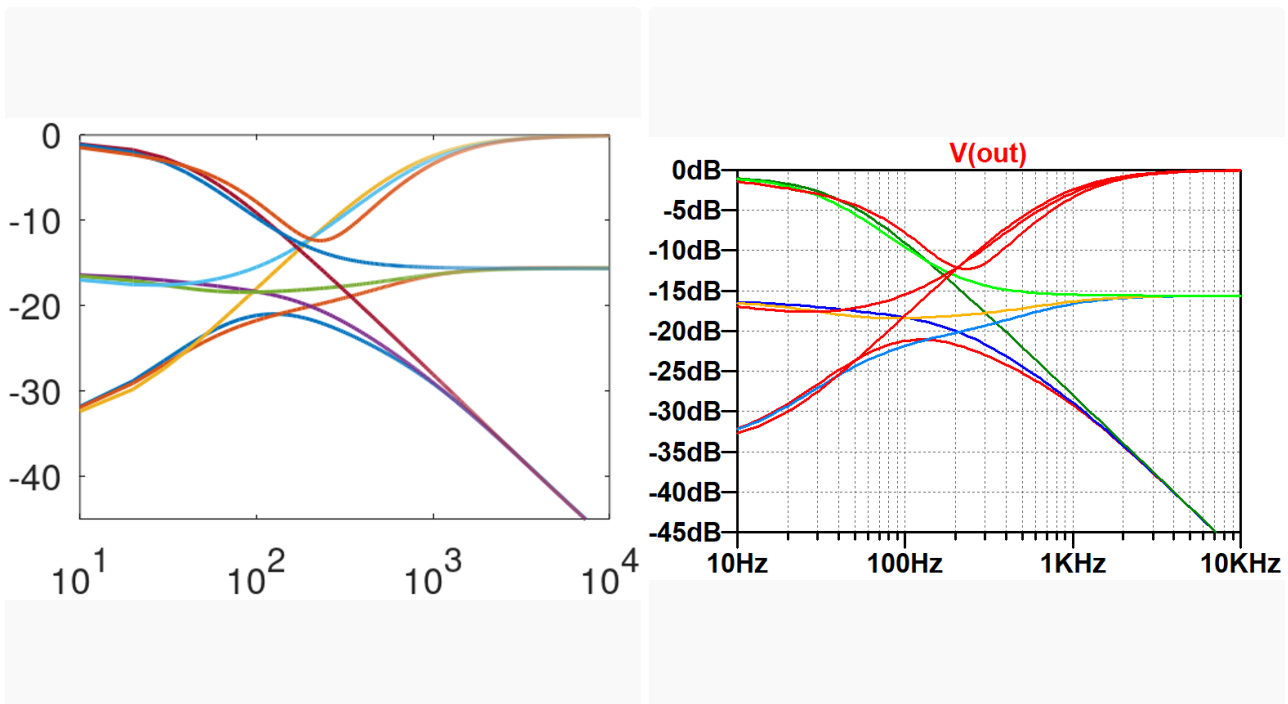
This is the formula we use to actually code the digital filter in C++ or the Rust programming language. Specifically, the current output sample is

$$y[n] = \frac{1}{A_0} [B_0x[n] + B_1x[n - 1] + B_2x[n - 2] + B_3x[n - 3] + B_4x[n - 4]$$

$$- A_1y[n - 1] - A_2y[n - 2] - A_3y[n - 3] - A_4y[n - 4]]$$

The filter has 8 coefficients that change when the guitar player rotates a knob. When that happens, the analog coefficients $a_0 \dots a_4$ and $b_0 \dots b_4$ are recalculated and used to recompute the digital coefficients $A_0 \dots A_4$ and $B_0 \dots B_4$. The new digital coefficients are then used to compute the output samples $y[n]$. In the Line 6 AX2, for example, a microcontroller detects a change in a knob position, computes the updated digital coefficients, and sends them to a digital signal processor that uses them to compute the output samples.⁵

Here is the frequency response of the digital filter plotted using GNU Octave. To facilitate a direct comparison, the SPICE simulation for the original analog filter is shown in the next plot.



The results are quite accurate for the relatively high 48kHz sample rate. Some deviation at high frequencies can be expected for lower sample rates.

References

- 1Fractal Audio Systems, **Multipoint Iterative Matching and Impedance Correction Technology (MIMICTM)**, April 2013, p. 7.
- 2David T. Yeh and Julius O. Smith, "Discretization of the '59 Fender Bassman Tone Stack," International Conference on Digital Audio Effects (DAFx-06), 2006.
- 3Richard Kuehnel, [Guitar Amplifier Electronics: Circuit Simulation](#), (Seattle: Amp Books, 2019).
- 4Fractal Audio Systems, **Multipoint Iterative Matching and Impedance Correction Technology (MIMICTM)**, April 2013, p. 7.
- 5U.S. Patent 5,789,689.